



Université Paris-Saclay – UVSQ
Master 2 DataScale
Module : Gestion de Processus Métiers

RAPPORT DE PROJET

TD3 : Implémentation du Processus d'Évaluation des Demandes de Prêt en Microservices

Étudiants	MBOUP EI Hadji & NDIAYE Pape Mor
Formation	Master 2 DataScale – Paris-Saclay
Modules	BPM & CLOUD
Enseignant	Yehia Taher
Technologies	Python, FastAPI, RabbitMQ, Celery, Redis, Docker, Kubernetes/minikube
Date	Mars 2026

Lien github : https://github.com/Elmboup/Loan_microservices.git

Table des Matières

Table des Matières	2
1. Introduction et Contexte du Projet	3
2. Analyse du Processus Métier (BPMN).....	3
2.1 Diagramme BPMN de référence	3
2.2 Analyse des flux principaux.....	4
2.3 Points clés d'implémentation issus du BPMN	4
3. Architecture du Système (Exercice 1)	4
3.1 Décomposition en microservices	5
3.2 Infrastructure technique	6
3.3 Modélisation de la communication inter-services.....	6
4. Flux Métier Détaillé et Événements	6
4.1 Séquence des événements.....	6
4.2 Catalogue des routing keys.....	7
5. Tâches Asynchrones et Celery (Exercice 2)	7
5.1 Tâches Celery implémentées.....	7
5.2 Configuration technique	7
5.3 Monitoring avec Flower	8
6. Messagerie Fiable et Pattern Saga (Exercice 3)	8
6.1 Durabilité des messages.....	8
6.2 Pattern Saga – Compensation en cas d'échec	9
7. Notifications Temps Réel et Dashboard (Exercice 4).....	9
7.1 Server-Sent Events (SSE)	9
7.2 WebSocket.....	9
7.3 Dashboard de suivi	9
8. Déploiement et Infrastructure	10
8.1 Docker Compose (environnement de développement).....	10
8.2 Kubernetes / Minikube (déploiement alternatif).....	10
8.3 URLs d'accès	10
9. Procédure de Test et Validation du Flux	10
9.1 Étape A – Création d'un dossier incomplet	10
9.2 Étape B – Complétion des documents.....	11
10. Choix Techniques et Justifications	12
10.1 Pourquoi FastAPI ?	12
10.2 Pourquoi RabbitMQ plutôt que Kafka ?.....	12
10.3 Pourquoi Celery + Redis ?	13
10.4 Alignement avec le BPMN	13
11. Limites et Perspectives d'Amélioration	13
11.1 Limites actuelles.....	13
11.2 Pistes d'amélioration	13
12. Conclusion	13

1. Introduction et Contexte du Projet

Ce projet porte sur l'implémentation d'un processus métier complet de gestion des demandes de prêt immobilier en architecture microservices.

L'objectif principal est de mettre en œuvre des concepts de : décomposition fonctionnelle en services indépendants, communication événementielle asynchrone, tolérance aux pannes, et notification temps réel des clients. Ces problématiques sont au cœur des systèmes d'information modernes, notamment dans le secteur bancaire et financier.

Ce rapport documente l'ensemble des choix de conception et d'implémentation réalisés pour couvrir les quatre exercices du TD, en s'appuyant sur le diagramme BPMN fourni comme référence du processus métier.

Périmètre fonctionnel couvert

- Exercice 1 – Structuration en microservices et communication inter-services (REST + RabbitMQ)
- Exercice 2 – Tâches asynchrones avec Celery et monitoring via Flower
- Exercice 3 – Messagerie fiable, durabilité des messages et mécanisme de Saga (compensation)
- Exercice 4 – Notifications temps réel (SSE + WebSocket) et dashboard de suivi

2. Analyse du Processus Métier (BPMN)

Le processus de traitement d'une demande de prêt immobilier est modélisé selon la notation BPMN 2.0. Il implique quatre acteurs principaux (pools/lanes) : le Demandeur (Applicant), le Chargé de Prêt (Loan Officer), le Responsable Financier (Financial Officer) et l'Évaluateur Immobilier (Property Appraiser), ainsi qu'un représentant de l'assurance.

2.1 Diagramme BPMN de référence

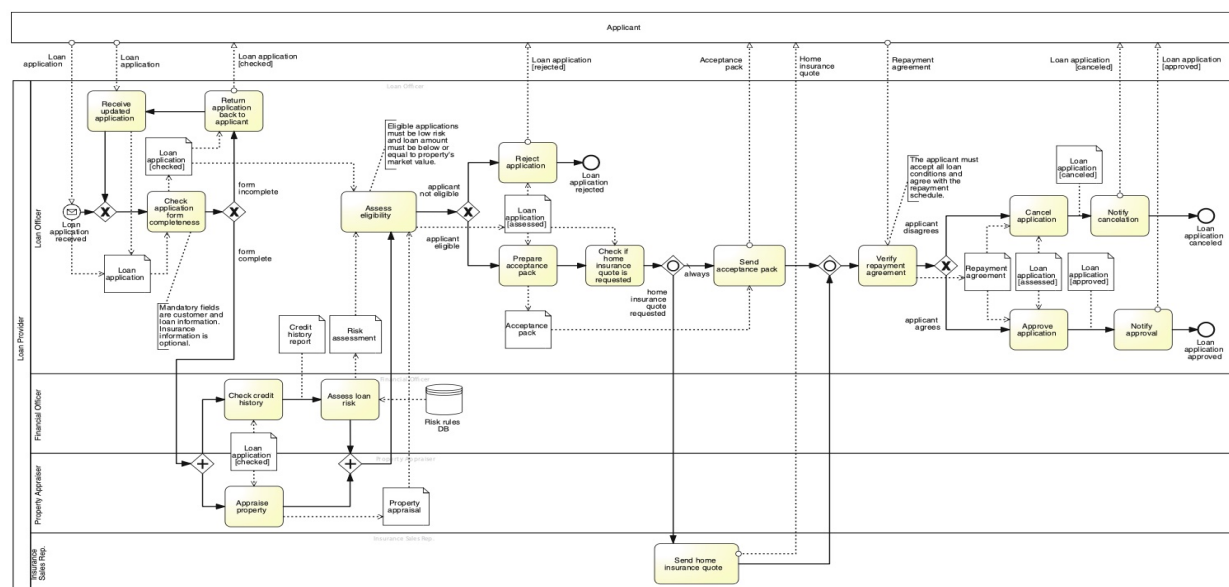


Figure 1 – Diagramme BPMN du processus d'évaluation des demandes de prêt immobilier

2.2 Analyse des flux principaux

Le processus se déroule selon les grandes phases suivantes :

1. Réception et vérification de la complétude du dossier par le chargé de prêt. Si incomplet, retour au demandeur pour compléments.
2. Vérification simultanée (parallèle) de l'historique de crédit (Financial Officer) et de l'évaluation du bien (Property Appraiser), déclenchée uniquement après réception d'un dossier complet.
3. Évaluation de l'éligibilité par le chargé de prêt (score crédit + valeur bien), produisant une décision : rejet ou préparation du dossier d'acceptation.
4. Si éligible : préparation du dossier d'acceptation, vérification optionnelle d'une assurance habitation, envoi du dossier au demandeur.
5. Collecte de l'accord ou du refus du demandeur sur les conditions (tableau de remboursement). Finalisation : approbation ou annulation du prêt.

2.3 Points clés d'implémentation issus du BPMN

Règle de synchronisation fondamentale

Les évaluations crédit et bien ne se déclenchent qu'après l'événement `loan.documents.received` (dossier complet reçu). Cette règle garantit qu'aucune ressource n'est consommée sur un dossier incomplet et correspond à la gateway parallèle (+) du BPMN.

3. Architecture du Système (Exercice 1)

L'architecture retenue est une architecture microservices événementielle (Event-Driven Architecture – EDA). Chaque service est un processus Python indépendant, exposant une API REST via FastAPI et communiquant avec les autres services exclusivement via des événements publiés sur RabbitMQ.

3.1 Décomposition en microservices

Service	Port	Responsabilité	Technologies clés
loan-service	8001	Création dossier, gestion documents, statut global	FastAPI, RabbitMQ publisher
credit-service	8002	Vérification crédit (asynchrone)	FastAPI, Celery, RabbitMQ
property-service	8003	Évaluation du bien immobilier (asynchrone)	FastAPI, Celery, RabbitMQ
decision-service	8004	Agrège crédit + bien, publie la décision	FastAPI, RabbitMQ consumer/publisher
insurance-service	8005	Génération devis assurance (asynchrone)	FastAPI, Celery, RabbitMQ
agreement-service	8006	Collecte accord/refus client final	FastAPI, RabbitMQ
notification-service	8007	SSE, WebSocket, dashboard HTML temps réel	FastAPI, SSE, WebSocket

Tableau 1 – Catalogue des microservices

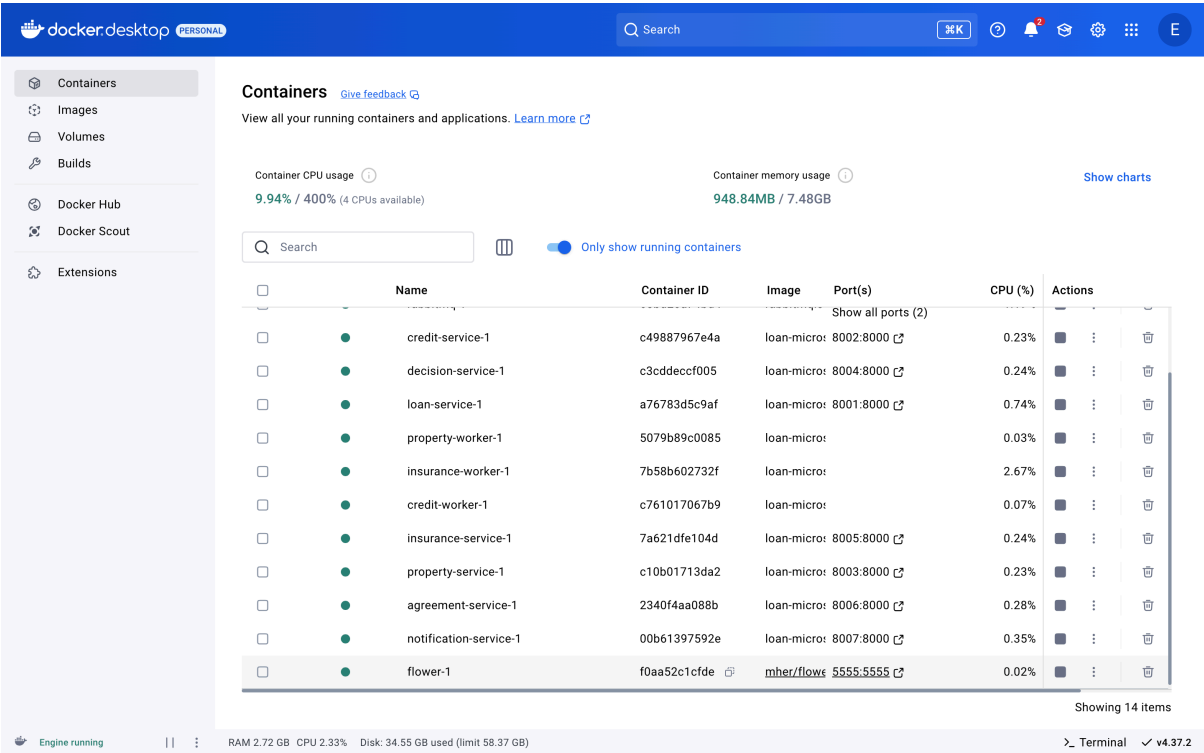


Image 1 – Catalogue des microservices sous Docker

3.2 Infrastructure technique

- RabbitMQ (broker d'événements) – échanges durables de type topic, port 5672, UI de gestion port 15672
- Redis – backend de résultats pour les workers Celery (persistance des états de tâches)
- Flower – interface de monitoring des workers Celery, port 5555
- Docker / Docker Compose – orchestration locale de l'ensemble des conteneurs
- Kubernetes (Minikube) – déploiement alternatif via manifests dans le dossier k8s/

3.3 Modélisation de la communication inter-services

Toute communication entre services passe par un exchange RabbitMQ de type topic nommé events. Chaque service publie des événements sur des routing keys spécifiques et consomme les événements qui le concernent via des queues dédiées. Cette approche garantit un découplage total : les services ne se connaissent pas directement.

Principe de l'enveloppe d'événement commune

Chaque message publié sur RabbitMQ respecte un format d'enveloppe standardisé incluant : event_id (UUID), event_type (routing key), timestamp (ISO 8601), loan_id (UUID), payload (données métier) et attempt (compteur de tentatives pour les retries).

4. Flux Métier Détaillé et Événements

Le flux événementiel implémenté reflète fidèlement la séquence du BPMN. La contrainte principale est que les traitements lourds (crédit, bien) ne démarrent qu'après confirmation de la complétude du dossier.

4.1 Séquence des événements

6. POST /loans → création du dossier → publication de loan.created
7. Si dossier incomplet → publication de loan.documents.requested, retour au client
8. POST /loans/{id}/documents → complément du dossier → publication de loan.documents.received
9. Déclenchement parallèle : credit-service (credit.checked) + property-service (property.evaluated)
10. decision-service agrège les deux résultats → publication de decision.made puis loan.approved ou loan.rejected
11. Si approuvé : acceptance.package.sent, puis collecte agreement via agreement-service
12. Finalisation : loan.final.approved (si accord) ou loan.cancelled (si refus/annulation)

4.2 Catalogue des routing keys

Routing Key	Direction	Description
<code>loan.created</code>	loan-svc → bus	Nouveau dossier créé
<code>loan.documents.requested</code>	loan-svc → bus	Documents manquants, retour client
<code>loan.documents.received</code>	loan-svc → bus	Dossier complet – déclencheur des évaluations
<code>credit.checked</code>	credit-svc → bus	Résultat de la vérification crédit
<code>property.evaluated</code>	property-svc → bus	Résultat de l'évaluation du bien
<code>decision.made</code>	decision-svc → bus	Décision d'éligibilité prise
<code>loan.approved</code>	decision-svc → bus	Prêt approuvé
<code>loan.rejected</code>	decision-svc → bus	Prêt rejeté
<code>credit.compensate</code>	decision-svc → bus	Ordre de compensation crédit
<code>credit.compensated</code>	credit-svc → bus	Compensation crédit effectuée
<code>acceptance.package.sent</code>	decision-svc → bus	Dossier d'acceptation envoyé
<code>agreement.received</code>	agreement-svc → bus	Réponse client reçue
<code>agreement.accepted</code>	agreement-svc → bus	Client accepte les conditions
<code>agreement.declined</code>	agreement-svc → bus	Client refuse les conditions
<code>loan.final.approved</code>	agreement-svc → bus	Prêt définitivement approuvé
<code>loan.cancelled</code>	agreement-svc → bus	Prêt annulé
<code>insurance.quote.ready</code>	insurance-svc → bus	Devis assurance disponible

Tableau 2 – Catalogue complet des événements RabbitMQ

5. Tâches Asynchrones et Celery (Exercice 2)

Certaines étapes du processus métier sont intrinsèquement longues : consultation des bases de scoring crédit, estimation immobilière, génération de documents. Exécuter ces opérations de manière synchrone bloquerait le thread HTTP et dégraderait fortement l'expérience utilisateur. Celery est utilisé pour les déléguer à des workers dédiés.

5.1 Tâches Celery implémentées

- Vérification du score de crédit (credit-service) – simule une consultation asynchrone d'une base de scoring, avec délai configurable.
- Évaluation immobilière (property-service) – modélise l'analyse de la valeur du bien, potentiellement dépendante d'APIs externes.
- Génération du devis d'assurance (insurance-service) – produit un devis asynchrone transmis ensuite via événement `insurance.quote.ready`.

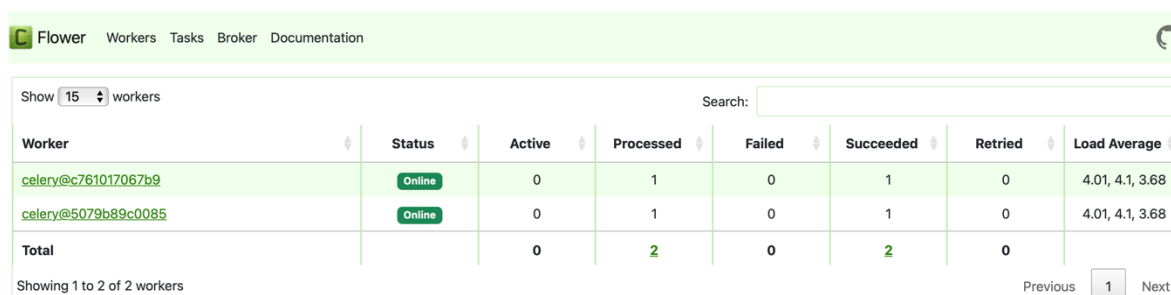
5.2 Configuration technique

Chaque service disposant de tâches asynchrones intègre un worker Celery configuré avec Redis comme backend de résultats. La configuration garantit :

- Persistance des états de tâches dans Redis (pending, started, success, failure)
- Retries automatiques en cas d'échec temporaire (jusqu'à 3 tentatives, délai exponentiel)
- Visibilité complète via Flower : état par worker, historique des tâches, temps d'exécution

5.3 Monitoring avec Flower

Flower est accessible sur le port 5555 et fournit une interface de monitoring en temps réel des workers et des tâches Celery. Il permet de surveiller le nombre de tâches actives, en attente, réussies ou en échec, ainsi que les performances par worker. Des endpoints /metrics exposent également des métriques compatibles Prometheus sur chaque service.



Worker	Status	Active	Processed	Failed	Succeeded	Retried	Load Average
celery@c761017067b9	Online	0	1	0	1	0	4.01, 4.1, 3.68
celery@5079b89c0085	Online	0	1	0	1	0	4.01, 4.1, 3.68
Total		0	2	0	2	0	

Image 2 – Interface flower (Tâches Celery)

6. Messagerie Fiable et Pattern Saga (Exercice 3)

La fiabilité des échanges entre services est critique dans un contexte financier. Une demande de prêt implique plusieurs étapes interdépendantes : si l'une échoue, les étapes précédentes doivent pouvoir être annulées. C'est l'objet du pattern Saga et des mécanismes de compensation.

6.1 Durabilité des messages

Paramètres de durabilité RabbitMQ

- Exchange durable de type topic nommé events – survit aux redémarrages de RabbitMQ
- Dead Letter Exchange (DLX) events.dlx – reçoit les messages en échec définitif
- Queues durables avec Dead Letter Queue (DLQ) par service : <service>.dlq
- Acknowledgement explicite (ack) – un message n'est supprimé de la queue qu'après traitement confirmé
- Champ attempt dans l'enveloppe – contrôle le nombre de tentatives (max 3). Au-delà : nack(requeue=False) vers la DLQ

6.2 Pattern Saga – Compensation en cas d'échec

Le Saga Pattern est implémenté pour gérer les transactions distribuées sans recourir à un gestionnaire de transactions global (2PC). En cas d'échec partiel, chaque service publie un événement de compensation que les services concernés consomment pour annuler leur action.

Scénario de compensation implémenté :

13. property-service publie property.evaluated avec property_ok = false (bien non éligible)
14. decision-service détecte l'échec et publie loan.rejected + credit.compensate
15. credit-service consomme credit.compensate, annule la réservation du scoring, publie credit.compensated
16. notification-service informe le client du rejet et de la raison

7. Notifications Temps Réel et Dashboard (Exercice 4)

Le notification-service (port 8007) centralise toutes les communications temps réel vers les clients et le personnel de suivi. Il combine deux mécanismes complémentaires : Server-Sent Events (SSE) pour les flux unidirectionnels et WebSocket pour les communications bidirectionnelles.

7.1 Server-Sent Events (SSE)

Le SSE permet d'envoyer des événements du serveur vers le client via une connexion HTTP persistante. Chaque fois qu'un événement RabbitMQ concernant un dossier est reçu, le notification-service le pousse immédiatement vers tous les clients SSE abonnés à ce dossier. L'endpoint GET /loans/{id}/stream permet au frontend de s'abonner au flux d'un dossier spécifique.

7.2 WebSocket

Le WebSocket offre une communication bidirectionnelle full-duplex. Il est utilisé pour des interactions plus riches, notamment permettre à un client de soumettre des décisions (accord/refus) en temps réel tout en recevant instantanément les confirmations du serveur.

7.3 Dashboard de suivi

Un dashboard HTML est servi statiquement sur <http://localhost:8007/>. Il affiche en temps réel l'état de toutes les demandes de prêt actives, le flux d'événements par dossier et les métriques globales du système (taux d'approbation, temps de traitement moyen). Le dashboard consomme le flux SSE pour se mettre à jour sans rechargement de page.

8. Déploiement et Infrastructure

8.1 Docker Compose (environnement de développement)

Chaque microservice dispose de son propre Dockerfile. Docker Compose orchestre l'ensemble : RabbitMQ, Redis, les 7 microservices FastAPI, leurs workers Celery respectifs et Flower. Le démarrage complet s'effectue en une seule commande :

Commandes de démarrage

Lancement : `docker compose up -d --build`

Arrêt : `docker compose down`

8.2 Kubernetes / Minikube (déploiement alternatif)

Un dossier `k8s/` contient les manifests Kubernetes prêts à l'emploi, organisés en Namespace `loan-app`. Chaque microservice est déployé dans un Pod séparé avec son Service associé. RabbitMQ et Redis sont également déployés comme Deployments indépendants.

- Démarrage : `minikube start` → `eval "$(minikube docker-env)"` → `cd k8s && ./deploy.sh`
- Vérification : `kubectl -n loan-app get pods,svc`
- Accès local via port-forward : `kubectl -n loan-app port-forward svc/loan-service 8001:8000`

8.3 URLs d'accès

Interface	URL
Dashboard temps réel	<code>http://localhost:8007/</code>
Loan Service API (Swagger)	<code>http://localhost:8001/docs</code>
Notification Service API	<code>http://localhost:8007/docs</code>
RabbitMQ Management UI	<code>http://localhost:15672</code> (guest/guest)
Flower (Celery Monitor)	<code>http://localhost:5555</code>

9. Procédure de Test et Validation du Flux

Le test de validation du flux complet s'effectue en deux étapes, reproduisant le scénario nominal : création d'un dossier incomplet, puis complément des documents.

9.1 Étape A – Création d'un dossier incomplet

Création du dossier via l'API REST du `loan-service` :

Commande curl – Création dossier

```
curl -s -X POST http://localhost:8001/loans \
-H 'Content-Type: application/json' \
-d '{"client_id":"client-flow-test","insurance_interest":true}'
```

Résultat attendu à ce stade : présence de `loan.created` et `loan.documents.requested` dans le flux d'événements. Absence de `credit.checked`, `property.evaluated` et `decision.made`.

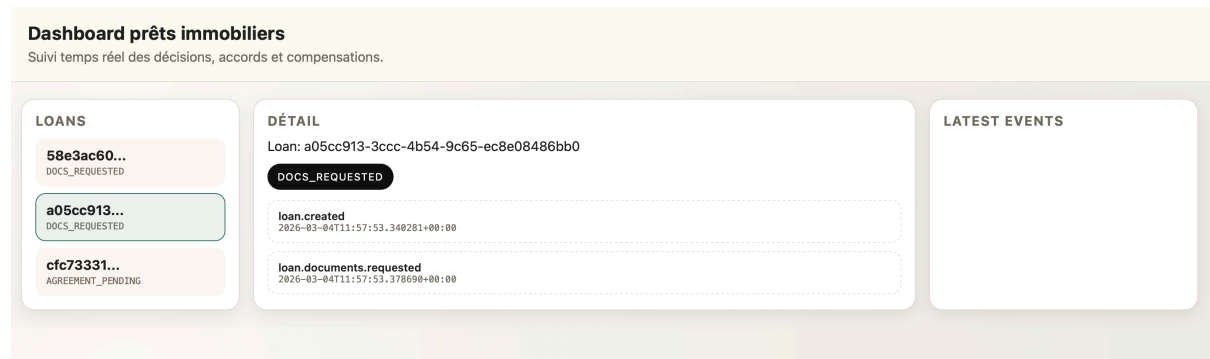


Image 3 – Résultats demande incomplète

9.2 Étape B – Complétion des documents

Commande curl – Ajout des documents

```
curl -s -X POST http://localhost:8001/loans/<loan_id>/documents \
-H 'Content-Type: application/json' \
-d '{"documents":{"id":"id.pdf","income_proof":"income.pdf","property_docs":"deed.pdf"}}'
```

Résultat attendu : apparition de `loan.documents.received`, puis déclenchement parallèle de `credit.checked` et `property.evaluated`, suivi de `decision.made` et `loan.approved` ou `loan.rejected`.

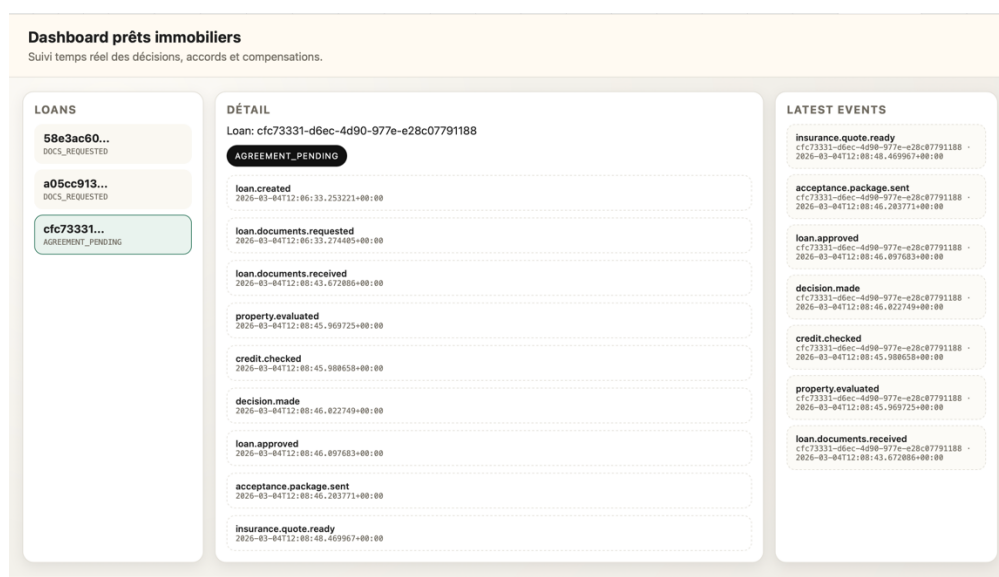


Image 4 – Résultats demande complète

10. Choix Techniques et Justifications

10.1 Pourquoi FastAPI ?

FastAPI a été retenu pour sa performance (basée sur Starlette/ASGI), son support natif des coroutines Python (async/await), et la génération automatique de la documentation Swagger/OpenAPI. Ces caractéristiques le rendent particulièrement adapté aux microservices nécessitant à la fois des endpoints REST synchrones et des flux SSE/WebSocket asynchrones.

loan-service 0.1.0 OAS 3.1
/openapi.json

default ^

GET	/health	Health	▼
POST	/loans	Create Loan	▼
POST	/loans/{loan_id}/documents	Upload Documents	▼
GET	/loans/{loan_id}	Get Loan	▼
GET	/metrics	Metrics	▼

Image 5 – Swagger interface

10.2 Pourquoi RabbitMQ plutôt que Kafka ?

RabbitMQ a été préféré à Kafka pour ce cas d'usage. Le volume de messages dans un processus de prêt est modéré, et RabbitMQ offre une gestion native des Dead Letter Queues, des échanges de type topic (routage par pattern) et des accusés de réception explicites (ack/nack) mieux adaptés à un workflow métier transactionnel. Kafka serait plus pertinent pour des volumes massifs ou des besoins de replay d'événements.

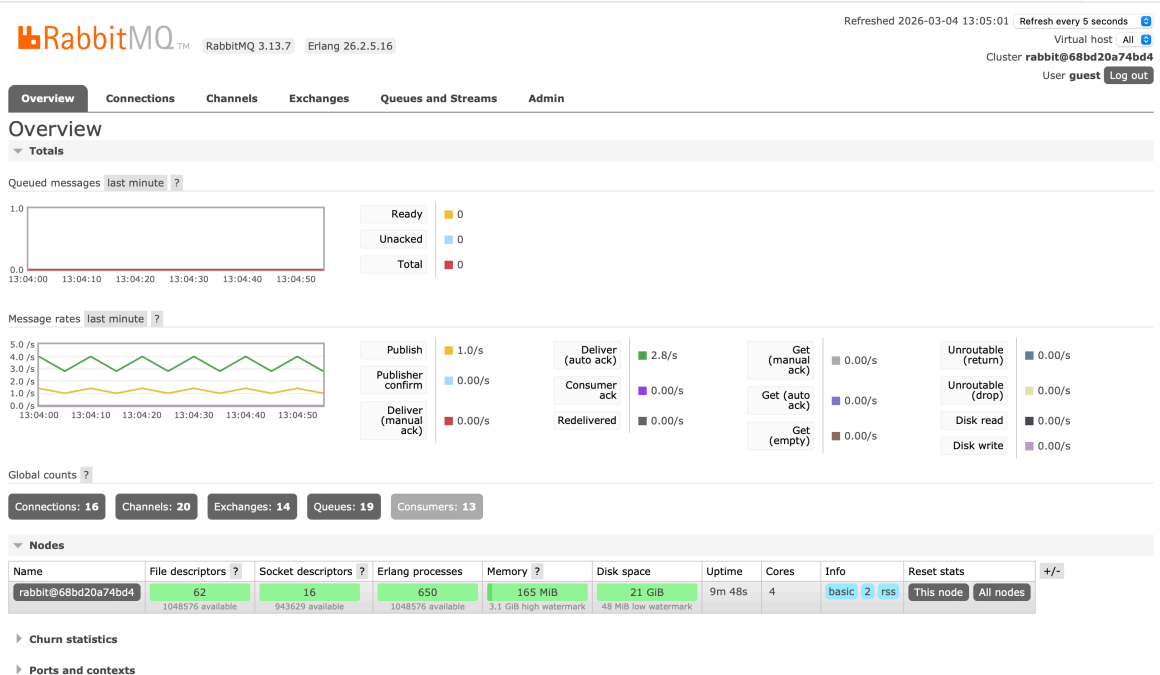


Image 6 – Notre interface RabbitMQ

10.3 Pourquoi Celery + Redis ?

Celery est la librairie de référence pour les tâches asynchrones en Python. Redis est utilisé comme broker léger et backend de résultats. Cette combinaison permet de déléguer facilement les traitements longs à des workers isolés, avec retry automatique et persistance des états, tout en conservant une configuration simple.

10.4 Alignement avec le BPMN

Un soin particulier a été apporté à l'alignement du flux événementiel avec le diagramme BPMN. La gateway parallèle (+) du BPMN (évaluation crédit et bien en parallèle) est reproduite par la publication simultanée des routing keys `credit.checked` et `property.evaluated`, consommées indépendamment par leurs services respectifs. Le decision-service joue le rôle de la gateway de synchronisation (join) en attendant les deux résultats avant de publier `decision.made`.

11. Limites et Perspectives d'Amélioration

11.1 Limites actuelles

- Stockage en mémoire – absence de base de données persistante. Les dossiers sont perdus au redémarrage des services.
- Pas de tests automatisés (pytest) – la validation repose sur des tests manuels via curl.
- Traitements simulés – les évaluations crédit et bien utilisent des `sleep()` pour simuler des délais, sans appel à de vraies APIs externes.
- Pas d'authentification – les endpoints REST ne sont pas sécurisés (absence de JWT ou OAuth2).

11.2 Pistes d'amélioration

- Ajouter PostgreSQL ou MongoDB pour la persistance des dossiers et événements.
- Implémenter une suite de tests avec pytest et pytest-asyncio, en mockant RabbitMQ.
- Intégrer un API Gateway (Kong, Traefik) pour centraliser l'authentification et le routage.
- Déploiement en production sur GKE (Google Kubernetes Engine) avec registre d'images, PersistentVolumes et autoscaling.
- Ajouter un service d'Event Sourcing pour rejouer l'historique complet d'un dossier.

12. Conclusion

Ce projet a permis d'implémenter un système de traitement de demandes de prêt immobilier complet, couvrant l'intégralité des quatre exercices du TD3. L'architecture microservices événementielle retenue garantit un découplage fort entre les services, une tolérance aux pannes via le pattern Saga, et une expérience utilisateur réactive grâce aux notifications temps réel.

Le choix de la stack FastAPI + RabbitMQ + Celery + Redis représente un compromis pertinent entre productivité de développement, performance et fiabilité opérationnelle. L'alignement soigneux avec le diagramme BPMN fourni assure que l'implémentation reflète fidèlement les exigences métier du processus de prêt.

Les limites identifiées (stockage en mémoire, absence de tests automatisés) constituent des axes d'amélioration naturels pour une mise en production. Le support Kubernetes via les manifests k8s/ démontre la capacité du système à évoluer vers un déploiement cloud.

*MBOUP El Hadji & NDIAYE Pape Mor – Master 2 DataScale, Université Paris-Saclay
Mars 2025*